

Chapter 2 : Java tutorial with examples demonstrating:

1. **Constructor/Method Overloading**
2. **Constructor Chaining**
3. **Static Variable**
4. **Static Method**

Each concept is illustrated using a basic "**Student Management System**" case study.

1. Constructor & Method Overloading

Case Study: A student can be created using different parameters:

- Default student with no details
- Student with only name
- Student with name and age

```
class Student {
    String name;
    int age;

    // Constructor Overloading
    Student() { // Default constructor
        this.name = "Unknown";
        this.age = 0;
    }

    Student(String name) { // Constructor with 1 parameter
        this.name = name;
        this.age = 0;
    }

    Student(String name, int age) { // Constructor with 2 parameters
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    // Method Overloading
    void updateDetails(String name) {
        this.name = name;
    }

    void updateDetails(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

public class OverloadingExample {
    public static void main(String[] args) {
        Student s1 = new Student();
    }
}
```

```

        Student s2 = new Student("Alice");
        Student s3 = new Student("Bob", 20);

        s1.display();
        s2.display();
        s3.display();

        // Method Overloading Example
        s3.updateDetails("Robert");
        s3.updateDetails("Robert", 22);
        s3.display();
    }
}

```

2. Constructor Chaining

Case Study: A student can be created using default values or with provided details, but all constructors call a base constructor.

```

class StudentChaining {
    String name;
    int age;

    // Constructor Chaining
    StudentChaining() {
        this("Unknown", 0); // Calls the two-parameter constructor
    }

    StudentChaining(String name) {
        this(name, 18); // Calls the two-parameter constructor
    }

    StudentChaining(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Student: " + name + ", Age: " + age);
    }
}

public class ConstructorChainingExample {
    public static void main(String[] args) {
        StudentChaining s1 = new StudentChaining();
        StudentChaining s2 = new StudentChaining("Charlie");
        StudentChaining s3 = new StudentChaining("Dave", 21);

        s1.display();
        s2.display();
        s3.display();
    }
}

```

3. Static Variable

Case Study: Every student belongs to the same college.

```
class StudentStatic {
    String name;
    int age;

    // Static Variable (Shared among all instances)
    static String college = "My University";

    StudentStatic(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age + ", College: " + college);
    }
}

public class StaticVariableExample {
    public static void main(String[] args) {
        StudentStatic s1 = new StudentStatic("Eve", 19);
        StudentStatic s2 = new StudentStatic("Frank", 22);

        s1.display();
        s2.display();

        // Changing the static variable affects all instances
        StudentStatic.college = "Your Institute";
        s1.display();
        s2.display();
    }
}
```

4. Static Method

Case Study: A static method can be used to print the total number of students.

```
class StudentStaticMethod {
    String name;
    int age;
    static int studentCount = 0; // Static Variable

    StudentStaticMethod(String name, int age) {
        this.name = name;
        this.age = age;
        studentCount++; // Increments for each student created
    }

    static void printTotalStudents() { // Static Method
        System.out.println("Total Students: " + studentCount);
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
```

```
public class StaticMethodExample {  
    public static void main(String[] args) {  
        StudentStaticMethod s1 = new StudentStaticMethod("George", 20);  
        StudentStaticMethod s2 = new StudentStaticMethod("Helen", 22);  
  
        s1.display();  
        s2.display();  
  
        // Calling the static method  
        StudentStaticMethod.printTotalStudents();  
    }  
}
```

Summary:

1. **Constructor/Method Overloading** → Multiple ways to initialize an object or update its details.
2. **Constructor Chaining** → One constructor calls another to avoid code duplication.
3. **Static Variable** → A common variable shared across all instances (e.g., college name).
4. **Static Method** → A method that belongs to the class, not an instance, and can manipulate static variables.

This covers the basics with simple case studies